

CS120 Project Report

Zheng Danjie, zhengdj2022@shanghaitech.edu.cn

Zhang Mingyu, zhangmy2022@shanghaitech.edu.cn

Project 0 & 1

Project 0 involves exploring basic audio device management and using certain applications to generate or receive audio signals, which uses the JUCE application and the ASIO sound driver.

A JUCE project involves using JUCE to select appropriate models to be used, and adding ASIO libraries for sound transmission.

Project 1 revisits project 0's setup, and is attached with 2 more tasks:

1. Generating sound waves. This helps emphasize understanding of audio sampling frequency.
2. Transmitting a binary file by sound modulation and demodulation. This aims to interpret sound wave information, also building a modulation method and frame structure design for a robust transmission.

For task 1, we followed JUCE's audio demo to generate sine waves and overlaid the acoustic waves for output.

For task 2, We managed to implement CRC calculation function, as well as modulation and demodulation method similar to provided MATLAB example code. However, we failed to end up with satisfying results. It's either caused by a noisy environment, the two nodes have a poor sound receiving quality, or we simply did not call relevant C++ libraries.

Project 2

Project 2 involves using cable connection to replace simple sound transmission in project 1, also adding an acknowledge system for signal transmission.

Beginning at project 2, we started to find potential solution on the Internet, as the problem of not able to call certain libraries and not able to use JUCE's functions still puzzles us.

We came across a github repository of previous CS120 course participants, which uses JUCE, VS2022 and C++ for projects. Our plan was to use their code for understanding,

test and modify it with correct running environment, and hand in as final result.

However, as we did not contact them in advance, we used a python accuracy check tool shipped with the repository and found an astonishing accuracy of around 5%, which was explained when we contact them after the project ended: the number 5.87 we ran from python test actually stands for 95% accuracy. But they also stated that their result at the time they did the project was actually 100%, which might have caused by interference during cable transferring. Also, they used MacOS for their project, which might have incompatible running environment comparing to our Windows environment.

As we did not know these details, we failed the project.

Project 3

Project 3 involves using sound cable to build a toy network, and shall be able to realize ICMP echo, router configuration and NAT implementation. This requires understanding echo requests and replies, managing IP datagram forwarding and network traffic, and modifying IP datagram for external communication.

Our implementation can handle several connections and echoes. However, due to an implementation fault, the connection from NODE2 to NODE1 was not successful. This leads to failing all tasks as we are out of depth to fix it.

This implementation was also planned to modify external code and hand in. To achieve this, we had several C++ libraries installed, such as *boost* and *npcap*. Sadly due to our timetable, we did not understand how external libraries' bugs can be fixed.

Also, other courses' final projects are entering proposal stage or ending, so we did not have time to debug or query the source code authors' for advice for testing even after project 3 ended.

Project 4

Project 4 extends from project 3, which requires upgrading the toy network to have DNS capability. The project uses NAT implemented in project 3 to configure a local DNS server on a certain node, enabling it to resolve domain names.

Since we did not finish project 3, we cannot complete project 4.

Reflection

From the github issue raiser of our class, it seems that not many uses C++ to complete the projects, even in the past semesters. And it seems reasonable, as libraries are not called as straightforward as, say, Python.

To put simply, using Visual Studio for coding is a true torture, as configurations must be set correctly, including existing libraries must involve path addition in key places such as linker.

To be honest, the two persons in this group has bad coding ability, and is sure to be unable to complete such complex C++ programming. The reason why we still chose to use JUCE, VS 2022 and C++, was that it was the only method that has official tutorial for setting up basic environment, such as win_asio_juce.pdf provided in github repository of our class.

We did think about changing the programming language, though. However, we are not sure what will happen if we use Python/MATLAB's libraries, and we are worried if undefined behavior will surface and cause large amount of debug time. As for Rust or Java, our semester is already full of courses, and we are clear of our poor self-learning ability that we cannot learn a new language in that depth to completing the projects. Worse still, none of these provides basic official support such as which libraries shall be installed.

Maybe our programming skill level was far from demand to complete the projects, but I still need to raise a suggestion that the project was too open for programming language to use, too limited for teaching basic utility usage (such as crucial functions in libraries), and a bit irrelevant to contents taught in course (as the projects seem to focus on how you can complete with master coding techniques, not the basic knowledge such as CRC, ACK, IP datagram, NAT, DNS and so on).